به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴) ۳۱ اردیبهشت ۹۸

دانشگاه صنعتی شریف مدرس: مهدی صفرنژاد

گراف

سؤال ۱. [۱۵ نمره] گراف جهت‌دار و وزن‌دار G را با یال‌های E، رئوس V و وزن‌های w داریم. توجه کنید وزن یال‌ها می‌تواند مثبت یا منفی باشد. برای یک عدد k مشخص، به مسیری بین دو راس u و v که کمترین مجموع وزن یال‌ها را دارد و حداکثر از k یال تشکیل شده است، «بهترین مسیر از u به v» می‌گوییم. رأس s و عدد k به ما داده شده است. الگوریتمی طراحی کنید که بهترین مسیر بین راس s و بقیهٔ رئوس را در $O(|V| + k|E|)$ پیدا کند.

پاسخ: الگوریتم بلمن فورد را اجرا می‌کنیم، با این تفاوت که بعد از k مرحله آن را متوقف می‌کنیم. در این صورت $O(|V|)$ برای مقداردهی اولیه و $O(k|E|)$ برای k دور اجرای حلقهٔ داخلی بلمن‌فورد صرف می‌شود که در کل محدودیت زمانی سوال را برآورده می‌کند. درستی الگوریتم با استقرا روی طول کوتاه‌ترین مسیر به دست می‌آید. به طور خلاصه دنبالهٔ کوتاهترین مسیر از u به v که طبق فرض سؤال به طول حداکثر k را درنظر بگیرید. در دور iاُم اجرای الگوریتم، مقدار d مربوط به رأس iاُم این دنباله به برابر با مقدار درست آن است که درستی الگوریتم را نتیجه می‌دهد.

سؤال ۲. [۱۸ نمره] برای هر یک از قسمت‌های این سوال، مسئله را به کمک گراف مدل کرده و سپس به کمک الگوریتم مناسب حل کنید. جزئیات مدل و دلیل درستی الگوریتم خود را شرح دهید.

آ. [۶ نمره] در یک کشور بین برخی از شهرها جاده وجود دارد به طوری که از هر شهر می‌توان به شهرهای دیگر رفت. این جاده‌ها می‌توانند یک طرفه یا دو طرفه باشند. همچنین هنگام ورود به هر شهر باید مالیاتی مانند $r > 0$ پرداخت شود. در این مسئله یک مسافر می‌خواهد از شهر A به شهر B برود و کمترین هزینه را پرداخت کند. برای یافتن مسیر بهینه به او کمک کنید.

پاسخ: برای هر شهر یک رأس در نظر می‌گیریم. اگر از شهر A به شهر B جاده‌ای وجود داشته باشد، از رأس متناظر A به راس متناظر B یک یال جهت‌دار رسم می‌کنیم. اگر هم بین دو شهر جاده دوطرفه باشد، یک یال جهت‌دار از A به B و یک یال جهت‌دار هم از B به A رسم می‌کنیم. حال به یک مسئلهٔ جست‌وجو در گراف رسیده‌ایم. اما در این مسئله رأس‌ها وزن دارند و نه یال‌ها. وزن‌های رأس‌ها را نیز قدرمطلق مالیات آن شهر می‌گیریم. برای این‌که وزن‌ها را از رأس‌ها به یال‌ها منتقل کنیم، کافی است به تمام یال‌های ورودی هر شهر، وزن آن شهر را نسبت دهیم. پس به یک مسئله ساده جست‌وجو کوتاه‌ترین مسیر در گراف وزن‌دار (با وزن نامنفی) رسیده‌ایم که به کمک الگوریتم دایسترا قابل حل است.

ب. [۱۲ نمره] یک شیمی‌دان می‌تواند در هر فرآیند یک ماده را به ماده‌ای دیگر تبدیل کند. به صورت دقیق‌تر برای برخی از مواد این شیمی‌دان می‌داند که می‌توان یک گرم از ماده اول را به r گرم ($r > 0$) از ماده دوم (در یک مرحله) تبدیل کند. او می‌خواهد بداند که مستقل از تعداد آزمایش‌ها، یک گرم مس را حداکثر به چند گرم طلا می‌توان تبدیل کرد.

پاسخ: برای هر ماده یک رأس در نظر می‌گیریم. همچنین اگر یک گرم از مادهٔ A در یک آزمایش به r گرم از مادهٔ B تبدیل شود، آنگاه یک یال جهت‌دار از A به B با وزن $-\log r$ رسم می‌کنیم. ادعا می‌کنیم مسئله به یافتن کوتاه‌ترین

مسیر از مس به طلا در این گراف وزن‌دار تبدیل شده است. برای این موضوع کافی است توجه کنیم که کوتاه‌ترین مسیر در حقیقت $\sum_i \log r_i = \log(\prod_i r_i) - \sum_i \log r_i$ را بین این دو رأس کمینه می‌کند و به عبارت دیگر را بیشینه می‌کند. پس مسیری را خروجی می‌دهد که $\prod_i r_i$ را بیشینه می‌کند که برابر با مقدار طلایی هست که از یک گرم مس در یک مسیر حاصل می‌شود که بیشینه کردن این مقدار برابر با پیدا کردن مسیری است که بیشترین طلا را از یک گرم مس حاصل می‌کند. اگر w وزن کوتاه‌ترین مسیر باشد، مقدار بیشترین طلای حاصل شده ۲^{-w} است.

برای حل این مسئله گراف می‌توانیم از الگوریتم بلمن‌ـ فورد استفاده کنیم. توجه کنیم که یال‌ها در این مسئله می‌توانند وزن منفی داشته باشند. حال اگر مسیری از مس به طلا نباشد، نمی‌توان چنین تبدیلی انجام داد. همچنین اگر بتوان با دور منفی به این طلا رسید، مقدار طلایی که می‌توان تبدیل کرد نامحدود خواهد بود و در غیر این صورت هم که می‌توانیم ۲^{-w} را خروجی دهیم. (البته توجه کنیم که اگر قانون بقای جرم را در حل مسئله وارد کنیم، می‌توانیم نتیجه بگیریم که دور منفی در این گراف وجود ندارد.)

سؤال ۳. **[۳۲ نمره]** دزدی پس از دستبرد زدن به یک بانک می‌خواهد به نقطهٔ امنی از شهر برای پنهان شدن برود به طوری که توسط پلیس دستگیر نشود. فرض کنید نقطهٔ شروع او s و نقطهٔ هدف او f باشد. او گراف نقاط مختلف شهر را در اختیار دارد. یال‌های این گراف نشان‌دهندهٔ خیابان‌ها هستند و وزن یک یال نشان‌دهندهٔ هزینهٔ مورد نیاز برای طی کردن آن خیابان است که عددی مثبت است. برخی از نقاط این گراف برای دزد مخفیگاه هستند و پلیس در این مکان‌ها نمی‌تواند دزد را شناسایی کند. پلیس هر سه دقیقه یک بار به نیروهای خود در سطح شهر دستور بازرسی افراد را می‌دهد. طی کردن هر یال برای دزد یک دقیقه طول می‌کشد. دزد برای این که توسط پلیس شناسایی نشود باید هر سه دقیقه یک بار زمانی که پلیس دستور بازرسی می‌دهد در یکی از مخفیگاه‌ها باشد. دقت کنید که نقطه‌ی هدف نیز یک مخفیگاه است.

آ. **[۱۲ نمره]** فرض کنید شهر به صورت یک جدول $m \times n$ باشد که خانه‌های آن رأس‌های گراف هستند و هر خانه به خانه‌های مجاور ضلعی خود یال دارد. هر خانه را با (x, y) نشان می‌دهیم. نقطهٔ s در جنوب غربی شهر در خانهٔ $(۰, ۰)$ قرار دارد و نقطهٔ f در شمال شرقی شهر در خانهٔ (m, n) قرار دارد. هر دقیقه دزد یا یک خانه به سمت شمال و یا یک خانه به سمت شرق می‌تواند حرکت کند. دقت کنید که دزد نمی‌تواند بعد از یک دقیقه در همان خانه باقی بماند و حتما باید حرکت کند. دزد چگونه می‌تواند مسیر با کمترین هزینه را از s به f پیدا کند به طوری که توسط پلیس دستگیر نشود؟ زمان الگوریتم شما باید $O(|V| + |E|)$ باشد.

ب. **[۲۰ نمره]** حال مسئله را در حالت کلی برای گراف دلخواه G حل کنید. در این جا فرض کنید که دزد می‌تواند برای مدت دلخواهی بر حسب دقیقه در یک رأس بماند و حرکتی نکند. الگوریتمی با زمان $O(|V| \log |V| + |E|)$ برای یافتن کم‌هزینه‌ترین مسیر از s به f ارائه دهید به طوری که دزد توسط پلیس دستگیر نشود.

راهنمایی: گرافی تشکیل دهید که رئوس آن موئلفه‌های دیگری علاوه بر مکان دزد داشته باشند.

پاسخ:

آ. از الگوریتم BFS با Branch and Bound استفاده می‌کنیم. به این صورت که سه لایه در BFS جلو می‌رویم، با نگاه داشتن طول کوتاه‌ترین مسیر که به یک مخفیگاه مشخص در لایه‌ی سوم ختم می‌شود. این الگوریتم را تا جایی ادامه می‌دهیم که به f برسیم.

ب. سه گراف $G_۱$ و $G_۲$ و H را به این صورت می‌سازیم: برای هر رأس v در گراف اصلی یک رأس $v_۱$ در $G_۱$ و یک رأس $v_۲$ در $G_۲$ در نظر می‌گیریم. همچنین برای هر مخفیگاه v در گراف اصلی یک رأس v_h در H در نظر می‌گیریم. برای هر یال $u\ v$ در گراف

اصلی یک یال از u_1 به v_2 در نظر می‌گیریم. در صورتی که v یک مخفیگاه باشد یک یال از u_2 به v_h در نظر می‌گیریم. در صورتی که u یک مخفیگاه باشد یک یال از u_h به v_1 در نظر می‌گیریم. یک رأس جدید به نام F در نظر می‌گیریم و رئوس f_1 و f_2 و f_h را با یال به طول ۰ به آن وصل می‌کنیم.

حال کافی است کوتاه‌ترین مسیر را از s_1 به F پیدا کنیم.

سؤال ۴. [۲۰ نمره] مؤلفه‌های قویاً همبند

در این سؤال باید تعداد و اندازهٔ مؤلفه‌های قویاً همبند (یا همبند قوی) یک گراف جهت‌دار را بیابید. برای این کار می‌توانید از الگوریتم Kosaraju استفاده کنید. مراحل این الگوریتم به شکل خلاصه عبارتند از

۱. ابتدا رأس‌ها را همانند روش مرتب‌سازی توپولوژیک بر حسب عکس زمان پایان DFSشان مرتب می‌کنیم (توجه کنید گراف لزوماً DAG نیست ولی با همان روش یک ترتیب برای راس‌ها به‌دست می‌آید).

۲. برحسب ترتیب به دست آمده در مرحلهٔ قبل شروع به اجرای DFS می‌کنیم اما در این مرحله یال‌های گراف را در جهت عکس درنظر می‌گیریم. رأس‌هایی که در این DFS از آن‌ها عبور می‌کنیم همگی در یک مؤلفهٔ قویاً همبند هستند. سپس رأس‌های مؤلفهٔ پیدا شده را کنار گذاشته و مرحلهٔ ۲ را تکرار می‌کنیم تا همهٔ مؤلفه‌های قویاً همبند یکی یکی پیدا شوند.

برای مطالعهٔ بیشتر می‌توانید به این لینک مراجعه کنید.

ورودی:
در خط اول ورودی n تعداد رأس‌ها ($n \leq ۱۰^۶$) و m تعداد یال‌ها ($m \leq ۱۰^۶$) آمده است. در m خط بعدی در هر خط دو عدد $u_i\, v_i$ می‌آیند که نشان دهندهٔ وجود یال از رأس u_i به رأس v_i است. شمارهٔ راس‌ها از ۰ تا $n-۱$ است.

خروجی:
در خط اول تعداد مؤلفه‌های قویاً همبندی گراف را چاپ کنید. در خط دوم اندازهٔ مؤلفه های قویاً همبند را به ترتیب اندازه‌شان از کوچک به بزرگ چاپ کنید.

پاسخ: پس از اجرای الگوریتم Kosaraju به مؤلفه‌های قویاً همبند دست پیدا می‌کنید که با شماردن اندازهٔ مؤلفه‌ها جواب مورد نظر بدست می‌آید.

کد ضمیمه شده برای این سؤال از آقای رضا یاربخش است.

سؤال ۵. [۲۵ نمره] اخیراً مسعود دست از پژوهش برداشته و بازی کردن با مکعب روبیک را امری مفیدتر از کار خود یافته و فکر می‌کند حل کردن این پازل کمک بیشتری به بشریت خواهد کرد. او که اخیرا به حل مکعب روبیک اعتیاد پیدا کرده است، یک مکعب جیبیِ! ۲ × ۲ × ۲ خریده که وقتی بیکار است، آن را حل کند تا در مسابقه‌ای که در هفته‌های آتی دارد بتواند موفق‌تر ظاهر شود. از این‌رو او برای حل این مساله در کمتر از ۵ ثانیه، به یاری یک کامپیوتر و صد البته کسی که بتواند کد بزند نیاز دارد، از این‌رو طبق روال، شما را به مسعود معرفی کرده‌ایم. در کنار مستند سؤالات، یک پوشه با نام `rubik` قرار دارد که بازی روبیک در آن پیاده‌سازی شده‌است.

هدف شما پیدا کردن کوتاه‌ترین مسیر بین حالت ابتدایی و حالت نهایی با استفاده از الگوریتم BFS است. اما استفاده از BFS ساده به طوری که تا عمق ۱۴ (که قطر گراف حالت‌های روبیک است) برود، کند است و مدت زمان اجرای آن به دقیقه می‌کشد و از حافظه زیادی استفاده می‌کند. حال باید الگوریتمی پیاده‌سازی کنید که مساله را در کم‌تر از ۵ ثانیه حل کند.

به‌این منظور، باید تابع `shortest-path` که در فایل `solver.py` است را پیاده‌سازی کنید که دو حالت را به عنوان ورودی می‌گیرد و لیستی که شامل کوتاه‌ترین مسیر بین این دو وضعیت است را به عنوان خروجی برگرداند.

همچنین توضیحات بیشتر درباره پروژه را می‌توانید در فایل‌های `README.txt` ای که در پوشه مربوطه هست پیدا کنید. علاوه بر این، تعدادی unit test در فایل `test-solver.py` آمده است که می‌توانید برنامه خود را با کمک آن‌ها تست کنید. نحوهٔ ارسال این سوال در صفحهٔ کوئرای درس قابل مشاهده است.

پاسخ: فایل کد نمونه با نام solver.py ضمیمه شده است. کد منتخب این بخش از خانم زینب احیایی انتخاب شده‌است.

سؤال ۶. [۲۵ نمره] وحید که علاقهٔ بسیار زیادی به سرورهایش دارد تصمیم گرفته که یک سیستم حمل و نقل طراحی کند، اما از آن‌جایی که با سرورهای خود بسیار سرگرم شده شما باید بخشی از این سیستم را طراحی کنید. او از شما می‌خواهد که الگوریتم dijkstra را پیاده سازی کنید. برای این کار ابتدا یک فایل زیپ داده می‌شود. شما باید تابع dijkstra در `dijkstra.py` را کامل کنید. ورودی‌های این تابع شامل `weight` که یک تابع بوده و با گرفتن دو گره به عنوان ورودی وزن بین آنها را می‌دهد، `nodes` که مجموعه تمام گره‌ها است، `source` و `destination` که گره شروع و پایان است می‌باشد. خروجی این تابع باید یک چندتایی (tuple) از لیستی از گره‌های درون مسیر و تعداد گره‌هایی که بررسی شدند باشند. هر گره زمانی بررسی شده حساب می‌شود که فاصلهٔ آن از مبدا حساب شود. برای هر گره مانند `node` همسایه‌های این گره در `node.adj` قرار دارند همچنین برای تست می‌توانید از `dijkstra-test.py` استفاده کنید. (هنگام ارسال پاسخ فایل `dijkstra.py` را آپلود کنید همچنین می‌توانید از کلاس‌های تعریف شده درون کد های موجود در فایل مانند priority queue نیز استفاده کنید.)

پاسخ: پاسخ نمونه در فایل `dijkstra_sol.py` ضمیمه شده است.

موفق باشید